

BendTT: An Affine Dependent Type Theory

Victor Taelin
Higher Order Company
Rio de Janeiro, Brazil
taelin@higherorderco.com

AI DISCLOSURE. Bend and BendTT were designed by the human author. This paper was written by Claude Fable 5.1 from the author’s code and design choices, and reviewed by the author. The Lean mechanization was human-specified, AI-proven, and checked by Lean.

Abstract

BendTT is the type theory of the Bend programming language. It has one sort with `Type : Type`, no universe hierarchy, datatypes with no positivity restriction, and it is consistent. The reason is a usage discipline: a value is consumed at most once unless the *kind* of its type says otherwise. Every type has a kind `Kind(q)` over a quantity q ; a binder marked `+` may be consumed any number of times, and it forms only when its type’s kind is `Data = Kind( $\omega$ )`. A function type is never `Data`, and a datatype earns `Data` at every constructor. So no closure is ever copied, and every known paradox of `Type : Type` or of negative datatypes copies a closure. Recursion passes one syntactic descent over the definition’s own case tree. Erased code is free and may diverge; nothing promotes it to live. There is no unification and there are no tactics: a claim is an `assert`, a proof is the `def` that fills it, and the match is the eliminator. We state the calculus, show how each attack dies, and describe a Lean 4 mechanization of the all-affine fragment.

1 Introduction

Bend is a functional language with dependent types and a parallel runtime [28]. Its checker implements a small type theory, BendTT, and this paper is about the one thing in it that is not standard: BendTT keeps `Type : Type`, impredicative quantification and recursive datatypes with negative occurrences, and it stays consistent.

The engines of the classical paradoxes all copy a function: Girard’s paradox and its Hurkens form apply a function-typed value to itself [16, 19], and Curry’s paradox through a negative datatype applies a node’s field to a copy of the node [11]. Logics without contraction admit naive comprehension [17, 18, 29]. BendTT carries that idea into a dependent type theory, where it replaces the universe hierarchy and the positivity check: a live variable is consumed at most once, and a function is never an exception.

Affinity alone would make the language useless: a proof feeds one hypothesis to the induction hypothesis and to a lemma. Bend restores reuse through the kind of a type. A binder may be marked `+`, and then its type must have kind `Data`. A datatype declares its kind and the checker earns it at every constructor; a function type is `Type`; a proposition is `Data` because its evidence is erased. The license to copy is a property of a *type*, never a proof carried by a *term*: there is

$t, A, B ::=$	
<code>x, k</code>	variable, reference to a definition
<code>@q x: A -> B</code>	function type
<code>x => f, f(a)</code>	abstraction, application
<code>q x = v; b</code>	let (not recursive)
<code>Kind(g), Quant</code>	a kind, the sort of quantities
<code>&0 &1 &2, g <&> h</code>	quantity literals, the meet
<code>D<p...>, C{a...}</code>	family instance, constructor
<code>\{C: h; m\}, \{ }</code>	match: peel one constructor; empty match
<code>{a == b : T}, {==}</code>	propositional equality, reflexivity
<code>%e : P; f</code>	rewrite by e through motive P
<code>{x : T}, ?name</code>	annotation, hole

Figure 1. Terms, in the surface syntax. q is a quantity sigil: `-` for 0, none for 1, `+` for ω . A family D carries an unspellable set r of already-matched constructors. Kinds are terms: `Type = Kind(1)` and `Data = Kind(2)`.

no modality, no copy class, no clone function. This inverts quantitative type theory [4, 5, 22], where ω is the ordinary binder and consistency comes from the universe hierarchy the theory keeps. Here 1 is the ordinary binder and ω is a permission a type earns. One rule shows the difference: an argument that enters a `+` binder is counted once, where QTT scales its usage by ω ; Section 3.4 says what that costs. Everything else is chosen to be cheap: one bidirectional pass [13] with a usage counter per binder, no unification, one syntactic test per self-call, and a dead fragment that costs nothing, may diverge, and is erased before the runtime, which moves every value by default.

2 The Calculus

BendTT is one file of TypeScript, `bend2/bend.ts`: parser, printer, pattern flattener, evaluator and checker. Evaluation, conversion, typing and validation are about a thousand lines over one `Term` type whose binders are host functions [25]. Every rule below is a derivation comment beside its case in that file.

2.1 Terms, Quantities, Kinds

Figure 1 lists the term formers; types are terms. A binder carries a quantity $q \in \{0, 1, \omega\}$, spelled `-x`, `x` and `+x`: erased, affine, reusable. Every type has a *kind* `Kind(g)` where g is a quantity term: a literal, a variable of the sort `Quant`, or a meet $g \sqcap h$ (surface `<&>`). Conversion orders kinds by the quantity order, `Kind(g) ≤ Kind(h)` when $h \leq g$; `Data`

$$\begin{array}{c}
\frac{(x : q' A) \in \Gamma}{\Gamma \vdash_q x : A \triangleright x^q} \\
\\
\frac{\text{book}(k) : T \quad q = 1 \Rightarrow k \text{ filled, self-call descends}}{\Gamma \vdash_q k : T \triangleright \emptyset} \quad \frac{\Gamma \vdash_0 A : \text{Kind}(q') \triangleright \cdot \quad \Gamma, x : q' A \vdash_0 B : \text{Type} \triangleright \cdot}{\Gamma \vdash_q \forall^{q'} x : A. B : \text{Type} \triangleright \emptyset} \\
\\
\frac{\Gamma, x : q' A \vdash_q f : B \triangleright \pi \quad \pi(x) \leq q'}{\Gamma \vdash_q \lambda x. f : \forall^{q'} x : A. B \triangleright \pi \setminus x} \quad \frac{\Gamma \vdash_q f : \forall^{q'} x : A. B \triangleright \pi \quad \Gamma \vdash_{q' \triangleright q} a : A \triangleright \pi'}{\Gamma \vdash_q f a : B[x := a] \triangleright \pi + \pi'} \quad \frac{\Gamma \vdash_q t : A \triangleright \pi \quad A \leq T}{\Gamma \vdash_q t : T \triangleright \pi} \\
\\
\frac{C \notin r \quad q = 1 \Rightarrow q' \neq 0 \quad \Gamma \vdash_q h : \forall^{q_i, q'} x_i : F_i. P(C\{x_1 \dots\}) \triangleright \pi \quad \Gamma \vdash_q m : \forall^{q'} s : D^{r \cup \{C\}} p_1 \dots P \triangleright \pi'}{\Gamma \vdash_q \lambda\{C : h; m\} : \forall^{q'} s : D^r p_1 \dots P \triangleright \pi \sqcup \pi'} \\
\\
\frac{q = 1 \Rightarrow q' \neq 0 \quad D^r \text{ has no constructor left, or a live } (x : E) \in \Gamma \text{ with } E \text{ empty}}{\Gamma \vdash_q \lambda\{ \} : \forall^{q'} s : D^r p_1 \dots P \triangleright \emptyset} \\
\\
\frac{\Gamma \vdash_q e : \{a == b : A\} \triangleright \pi \quad \Gamma \vdash_0 P : \forall x : A. \{a == x : A\} \rightarrow \text{Type} \triangleright \cdot \quad P b e \leq T \quad \Gamma \vdash_q f : P a \{ == \} \triangleright \pi'}{\Gamma \vdash_q \%e : P; f : T \triangleright \pi + \pi'}
\end{array}$$

Figure 2. Typing, the rules that differ from the textbook. $\Gamma \vdash_q t : T \triangleright \pi$ reads: under the book and context Γ , at demand $q \in \{0, 1\}$, t has type T and consumes π , a map from variables to quantities. A dot is a measure the rule drops. $\pi + \pi'$ adds pointwise ($1 + 1 = \omega$), $\pi \sqcup \pi'$ takes the pointwise maximum, $q' \triangleright q$ is 0 when $q' = 0$ and q otherwise, and $r \cdot q'$ is 0, q' or $q' + q'$ as r is 0, 1 or ω . In REF, a live reference needs a filled or native body, and a self-reference needs its pending spine to descend (Section 2.5). In MAT, F_i and q_i are the constructor's fields at the family's parameters. The rest: a constructor $C\{a_1 \dots\}$ at $D^r p_1 \dots$ with $C \notin r$, and a family instance $D p_1 \dots$ at $\text{Kind}(G[p_1 \dots])$, check each argument at the gate like APP; a let $q' x = v$; b is $(\lambda x. b) v$ with v inferred and its type checked dead at $\text{Kind}(q')$; $\{a == b : A\} : \text{Data}$ with all three parts dead, and $\{ == \}$ proves it when $a \equiv b$; $\text{Kind}(g) : \text{Type}$ with g dead at Quant, and $g \sqcap h : \text{Quant}$ checks both sides at the ambient demand and adds their measures.

fits every kind, every kind fits Type, a kind fits a meet when it fits either side, a meet fits a kind only when both its sides do, and a stuck quantity fits only itself. The meet reduces only when forced: ω is its identity, 0 absorbs, two literals meet, and a stuck side stays stuck, so no declaration order can decide a meet early.

A *book* is an ordered list of declarations. `type D<p.> is Kind(G)`: declares a family with parameters and a kind, then its constructors, each a telescope of fields tipped at `D<p.>`. `assert k: T` declares a name at a closed type and a later `def k(x...)`: fills it. Until its fill a name is an *axiom*: it may appear in types and other dead positions, and live code may not consume it. A definition may reference itself only through the descent rule of Section 2.5, and never a later name, so the reference graph is acyclic and mutual recursion cannot split a loop across two definitions. The flattener compiles `match/case` blocks into a case tree of one-constructor peels [21], uncovered rows becoming the empty match. A match scrutinizes only a parameter or a field bound by an enclosing match; a computed scrutinee is an error that says to give it its own definition (Section 4).

2.2 Reduction and Conversion

Evaluation is weak head reduction. A lambda applied steps; a let substitutes; a reference unfolds only when its spine reaches its declared arity and it has a body, so an axiom and an underapplied definition are stuck values; a match meeting its constructor passes the fields to the arm, and any other constructor falls whole to the default m , whose domain records the peeled constructor in r ; a rewrite sheds when its evidence reaches `{ == }`. When a definition's match

sticks on a variable, the evaluator answers the definition applied to that variable, not the exposed case tree, so goals stay in the vocabulary of the source; this refolding is what lets an induction hypothesis match a goal (Section 4).

Conversion $A \leq B$ is reduce-and-compare at every node, up to η for functions, and it is a preorder, not an equality: directional at kinds and at match residuals (more constructors peeled fits fewer), domains of a function type swapped, quantities exact, and every part that flows both ways (an argument, a parameter, a field, an equation endpoint) compared as a symmetric $A \equiv B$. Reflexivity uses `{ == }` does not prove `{Data == Type : Type}`, which J could transport into a cast. Conversion may diverge on dead code; a hang accepts nothing (Section 3.4).

2.3 Typing

Figure 2 gives the rules. Inference synthesizes, checking pushes a goal into the introduction forms, and the two meet at conversion. Both directions carry a *demand* q : 0 checks a term dead, 1 live. There is no demand ω : a term checked at demand ω would let a binder inside it contract, and Atkey shows this also breaks substitution [4]. Sequential premises add their measures, the arms of a match join since one of them runs, and a binder validates $\pi(x) \leq q'$ when it closes: two live uses of an affine binder saturate to ω and fail there. A reference costs nothing: code is free.

The dead fragment is free. Types in premises, binder domains, equality endpoints and rewrite motives check at demand 0, where the measure is dropped. A dead term may mention consumed variables, may diverge, and may inhabit Empty. No rule coerces dead to live: an erased binder's uses

count at 0, and a match on an erased scrutinee is refused in a live region.

Reuse is gated, not scaled. An argument to a binder of quantity q' checks at demand $q' \triangleright q$, dead if the binder is erased and the ambient demand otherwise, and its measure adds once. So a value bound once may enter a $+$ binder, and the callee copies it: *certify-once*. It is licensed because the $+$ binder's domain checked against `Data` when the function type formed, and a `Data` value holds nothing affine (Section 2.4). A match hands each field out at the field's quantity times the scrutinee's, so a $+$ scrutinee makes its plain fields reusable.

Matching is consumption. A match is a value of function type, and applying it consumes the scrutinee. The arm receives the fields at a goal specialized to the rebuilt constructor, $P(C\{x_1, \dots\})$: dependent elimination with no generated eliminator and no unification. The default receives the scrutinee with C peeled onto r , and the empty match closes the chain when no constructor remains, or when a *live* binder in scope has an emptied type; an erased one proves nothing, since dead code inhabits it.

Equality is the $\mathcal{J}$ axiom with an explicit motive. From $e : \{a == b : A\}$, a rewrite maps a goal $P\ b\ e$ to the obligation $P\ a\ \{==\}$; the programmer writes P , and it checks dead. The evidence runs, so it checks at the ambient demand. An equation is `Data`: its evidence is erased, and a closed live proof normalizes to $\{==\}$, so copying it copies nothing.

2.4 Kinds Are Earned

A $+$ binder forms only over a `Data` type, so the security of the theory rests on what may be `Data`. The rules assign kinds by shape: a function type is `Type`, because a closure captures; an equation is `Data`; a family has the kind it declares, $\text{Kind}(G)$ over its parameters. The book validator earns the declaration: for each constructor it walks the telescope, parameters then fields, in the real constructor context, and checks a binder of quantity q against $\text{Kind}(q)$ and a *live* field against the declared $\text{Kind}(G)$; the tip must be the family applied to its own parameters, in order. From the base library:

```
type List<a, -A: Kind(a)> is Kind(a):
  Nil{}
  Con{head: A, tail: List<a, A>}

type Sigma<a, b, -A: Kind(a),
  -B: @-x: A -> Kind(b)>
  is Kind(a <&> b):
  Tuple{fst: A, snd: B{fst}}
```

A list is as reusable as its element; a pair is reusable when both halves are. `Nat`, `Bool`, `U32` and `String` are `Data`; `Array`, the IO handles and the effect type are `Type`. A recursive occurrence is assumed at the declared kind, with no fixed point: a live value is a finite tree, so the invariant that a `Data` value holds nothing affine follows by induction on the value. Values never weaken with their kinds: `List<&2, Nat>` and `List<&1, Nat>` are different types. Generic code takes the

quantity as an erased parameter (`forall -a: Quant`), under which $+$ is refused, since $\text{Kind}(a)$ does not reduce to `Data`; the license appears only after instantiation.

2.5 Descent

Self-reference passes one test, run against the definition's own case tree. Walking the body, the checker rebuilds the definition's left-hand side: a lambda binds the next *column*, a match peels the current column into a constructor of fresh field columns. At a live self-reference the pending arguments compare against the columns left to right, erased columns skipped: each must equal its column until one is a *strict subterm* of it, the same constructor with a strictly smaller field, or a term sitting inside one of the pattern's fields. Arguments past the decreasing one are free, so Ackermann passes: its inner call shrinks the first column, its outer call keeps it and shrinks the second. The comparison sees through lets, so `+p = p0` keeps `p` a subterm. No sizes are computed: this is a minimal member of the structural-recursion family [1, 15], chosen because it is one pass and trivial to audit. Dead demands skip the test, so a type may recurse freely; that is what makes $R = @-x: R \rightarrow \text{Empty}$ definable, and it is harmless because dead code never runs.

3 Why It Is Consistent

In a functional language there are two ways to loop: self-application and recursion. Descent closes the second. This section is about the first, and about why the copy license cannot be forged. Every example is a test in the repository, and the messages are the checker's own.

3.1 Self-Application Dies at the Counter

$\omega = (\lambda x. x\ x)(\lambda x. x\ x)$ uses its binder twice, so the measure saturates to ω , and a plain binder refuses it. The only binder that admits two uses is $+x$, and $+x$ forms only over a `Data` type. No function type is `Data`:

```
assert dupf:
  forall +f: Nat -> Nat
  Nat

- expected : Data
- observed : Type
```

Hurkens' paradox [19] is twenty definitions long in Bend and typechecks up to `tauinner`, the first place a function-typed variable is used twice; it dies there with `f` (consumed more than once). Impredicativity and `Type` : `Type` do no harm on their own.

3.2 Negative Types Are Legal, and Harmless

There is no positivity check. A HOAS-style term type is a fine declaration, and its evaluator is a fine program:

```

type Trm is Type:
  Lam{f: Trm -> Trm}
  Num{n: U32}

def app(t: Trm, x: Trm) -> Trm:
  match t:
    case Lam{f}:
      f(x)
    case Num{n}:
      Num{n}

```

Curry’s engine, `omega(Lam{f}) = f(Lam{f})`, needs the field `f` twice and dies at the counter like `dupf`. A `Data` declaration cannot hold the function either: `MkBox{f: Nat -> Nat}` under `type Box is Data` fails its field with `expected Data, observed Type`. So a negative field sits only in an affine type, where it is used at most once, and a value of a negative type is at worst an inert closure.

3.3 The License Cannot Be Forged

Every attack we know of tries to obtain `Data` for something affine. The *copy paradox* is the sharpest. In a theory whose contractible types are reusable, the type of copies of x , `&y: T -> {x == y : T}`, has the single inhabitant `(x, {==})`, and copying that pair copies x . Here it must be a datatype with a declared kind, and its live field `y: T` has kind `Type`, which `Data` does not admit: the declaration fails at its constructor. A *constructor-local quantity* fails the same way: a constructor that binds its own `L: Quant` and stores a field at `Kind(L)` is refused under `Data`, since a local `L` is never assumed to be `&2`. A *dead hypothesis* licenses nothing: an erased `-e: Empty` or `-c: Copiable(T)` in scope may never be supplied, and a `+` needs a kind that *reduces* to `Data`, which a type stuck on dead evidence never does. A *quantity equation* `{&1 == &2 : Quant}` is a legal type with no live proof, and a rewrite through it yields a value whose type is a stuck rewrite: nothing applies it. The *meet* charges both operands, so `q <&> &2` is not a free copy of `q`. A *quantity cast* fails at conversion: `@x: A -> B` and `@-x: A -> B` are different types. A *base name* cannot be refilled by a later file, and a *foreign fill* must answer the base library’s `!0` type, so no import line can spell a duplicator. A *circular fill* of `Forge(T): Data` is a live self-call with no shrinking column, refused by descent.

3.4 The Claims, and Their Price

For a book that validates, the theory claims subject reduction, progress, weak normalization of closed live terms, and no closed live inhabitant of `Empty`. The last three are stated at the live demand because dead code may rest on an axiom, diverge, or inhabit `Empty` by design: erased code is specification, not proof. Conversion, and so the checker, is a semi-decision procedure, as it already is in theories with divergent types [10]: a hang is “inconclusive”, never “accepted”. The stance is soundness on accept, not checker totality, in the line of NuPRL and Zombie [7, 9].

Certify-once has a price. Term-substitution reduction does not preserve the usage measure: unfold `f(+x)` at `f(y)` and `y` counts twice under its plain binder. So subject

reduction for usage is claimed for weak by-value reduction of closed terms, the only reduction the machine performs: when `f` unfolds, `y` is a value whose resources were consumed once, and the copy is `Data`, which owns nothing. Three invariants outside the checker carry this. No pass duplicates a term: the evaluator shares every argument, let value and field in a memoized cell, and the compiler is strict. A type with runtime ownership (a file, a socket, an array) is declared `Type`, never `Data`; this is a property of the base library, not a rule. A compiler may drop a copy the source spelled, never add one [28]. QTT avoids the question by scaling the argument’s measure, which it can afford because ω is its default; with 1 as the default, scaling would leave only closed data reusable.

One escape hatch exists and is always disclosed. A definition marked `@unsafe` skips descent and forms its `+` binders at any kind; the checker reports every book that uses one, so a clean report means none of the claims above is waived.

4 Proofs Without Tactics

A claim is an `assert` and a proof is the `def` that fills it. `forall` folds to a function type, `exists` to a dependent pair, `where` packs a hypothesis onto a binder; `{a != b : T}` is a function into `Empty`. An unfilled claim is a `TODO`: the checker counts it, the compiler refuses it, and no unproven name reaches live code.

The match is the eliminator. Matching a parameter refines the claim in each arm: the goal in the `!n+p` arm of a claim about `a` is the claim at `!n+p`, evaluated, with stuck self-calls refolded to source form. The induction hypothesis is the recursive call, and descent makes it valid. There is no unification, and there are no metavariables, implicit arguments or tactics: the assert of a helper is the motive of its match, which is why a computed scrutinee must go to its own definition. Rewriting is `J` with the motive written out: given `e : {a == b : T}`, the motive marks with `_` the places where `b` stands, the goal must be the motive at `b`, and the body proves it at `a`. So one states an equation with the term to eliminate on the right. The whole idiom, checked by Bend:

```

def add(a: Nat, b: Nat) -> Nat:
  match a:
    case 0n:
      b
    case 1n+p:
      1n+add(p, b)

assert zero:
  forall a: Nat
  {a == add(a, 0n) : Nat}

def zero(a):
  match a:
    case 0n:
      {==}
    case 1n+p:
      %zero(p) : {1n+p == 1n+_: Nat}
      {==}

assert comm:
  forall a: Nat
  forall +b: Nat
  {add(a, b) == add(b, a) : Nat}

def comm(a, b):
  match a:
    case 0n:
      zero(b)
    case 1n+p:
      %succ(b, p) : {1n+add(p, b) == _ : Nat}
      %comm(p, b) : {1n+add(p, b) == 1n+_ : Nat}
      {==}

```

In the successor arm of `zero` the goal evaluates to $\{1n+p == 1n+add(p, 0n)\}$; the hypothesis `zero(p)` proves $\{p == add(p, 0n)\}$, and the rewrite folds `add(p, 0n)` back into `p`, leaving reflexivity. `comm` needs `b` twice, in the lemma `succ` ($\{1n+add(a, b) == add(a, 1n+b)\}$, by the same induction) and in the hypothesis; `Nat` is `Data`, so `+b` licenses it. This is the idiom that affinity alone forbids and the kind restores. Statements are free: at demand 0 a theorem may quantify over functions and repeat variables at will. Affinity constrains what runs, never what is said.

5 Erasure and the Runtime

The checker elaborates as it checks: it returns the term with every node annotated by its type, and stores it for the compiler [28]. Erased binders, arguments and fields drop. Kinds, quantities, function types, family instances, equations and reflexivity become nothing; a rewrite compiles to its body. What survives is the live fragment: constructors with their live fields, lambdas over live binders, matches, and calls. A closed live proof is `{==}` and vanishes whole.

The usage discipline then pays twice. The checker needed only a counter per binder, threaded through the one pass it already makes. The runtime needs no garbage collector: a value has one owner, so a match frees its scrutinee as it opens it, arrays update in place, and a forked task carries no lock. Where a `+` licensed reuse, the compiler places a counted share or a borrow, and nowhere else. Reuse was derived from the kind of a type and never proved by a term, so the runtime never runs a copy the source did not spell.

Claim	Theorem
Confluence	<code>church_rosser_holds</code>
Subject reduction (weak, measure $\leq$)	<code>subject_reduction_holds</code>
Progress (live)	<code>progress_holds</code>
Normalization (weak, live)	<code>normalization_holds</code>
Consistency (live)	<code>consistency_holds</code>
The dead boundary, as a witness	<code>consistency_none_boundary</code>

Table 1. Claims and theorems in `bend.lean`, for the all-affine fragment.

6 The Mechanization

`bend2/bend.lean` mechanizes the core in Lean 4 [24], checked with a plain `lean` invocation: about twenty thousand lines, no `sorry`, no axiom declarations. Its first part is a specification that mirrors `bend.ts` section by section, in de Bruijn syntax; its second part proves the claims of Table 1.

The scope is exact and narrower than the language. The file mechanizes the *all-affine* fragment: no `+` binder, no `Data`, no kinds and no meet; ω exists only inside the measure, where it is always a violation. The `Data` layer of Section 2.4, certify-once included, is argued in Section 3.4 and audited by the tests; it is not a theorem. The file also lists the smaller places where the checker is more permissive than the model, such as the empty match under a live emptied binder and descent skipped at dead demand. Normalization and consistency are proven with recursive definitions included, by a Dershowitz–Manna multiset measure [12] over pending references. The boundary is a witness, not a caveat: in a well-formed book with a negative type, Curry’s self-application term checks *dead* at an empty family (`dead_omega_check`) and provably never live. The specification was written and reviewed by humans; the proofs were written by an AI system and are checked by Lean: trust the checker, audit the statements.

7 Discussion

What affinity takes away. Contraction on closures. The standard `map`, whose function is applied once per element, is ill-typed: `f` would need `+`, and a function type is never `Data`. Code is free, so a top-level definition may be called any number of times and maps specialized to a named function cover the common cases; but the closure-heavy style of Haskell does not transfer. Bend accepts this on purpose: the runtime wants the same restriction [28].

What it keeps. On the program side the baseline is C: first-order data, machine words, arrays updated in place, code called by name. That fragment passes through affinity untouched. On the proof side the reach is larger: statements are erased and cost nothing, and live proofs draw their reuse from `Data`, which is where induction lives. Nothing prevents

adding a universe hierarchy later; but Bend needs affinity anyway, and $\text{Type} : \text{Type}$ buys impredicative encodings and type-computing definitions no predicative hierarchy accepts.

Related work. Linear and dependent types meet in LLF [8] and in [20]; the quantitative line [4, 5, 22] and the graded line [2, 23] are closest in mechanism, and all keep a universe hierarchy. Recent linear dependent theories still add universe levels to avoid Girard’s paradox [14], or reach an impredicative universe through a model rather than $\text{Type} : \text{Type}$ [27]. To our knowledge none uses the absence of contraction for consistency. Among mechanized metatheories of practical kernels [3, 6, 26], ours proves normalization rather than assuming it.

Limitations. The consistency result is syntactic, relative to Lean’s own foundation, with no semantic model. The Data layer is argued and audited, not mechanized; formalizing it is the next planned extension. Equality is intensional, with no extensionality principle. And the theorems are about the calculus, not the code.

BendTT buys consistency with affinity instead of a universe hierarchy, earns reuse from the kind of a type instead of a proof, checks recursion with a descent rule an auditor can read in an afternoon, and erases everything that does not run. Nothing in it is difficult, and that is the point.

References

- [1] Andreas Abel and Thorsten Altenkirch. 2002. A Predicative Analysis of Structural Recursion. *Journal of Functional Programming* 12, 1 (2002), 1–41.
- [2] Andreas Abel, Nils Anders Danielsson, and Oskar Eriksson. 2023. A Graded Modal Dependent Type Theory with a Universe and Erasure, Formalized. *Proceedings of the ACM on Programming Languages* 7, ICFP (2023), 220:1–220:35.
- [3] Andreas Abel, Joakim Öhman, and Andrea Vezzosi. 2018. Decidability of Conversion for Type Theory in Type Theory. *Proceedings of the ACM on Programming Languages* 2, POPL (2018), 23:1–23:29.
- [4] Robert Atkey. 2018. Syntax and Semantics of Quantitative Type Theory. In *Proceedings of the 33rd Annual ACM/IEEE Symposium on Logic in Computer Science (LICS)*, 2018. 56–65.
- [5] Edwin Brady. 2021. Idris 2: Quantitative Type Theory in Practice. In *35th European Conference on Object-Oriented Programming (ECOOP) (LIPIcs)*, 2021. 9:1–9:26.
- [6] Mario Carneiro. 2024. Lean4Lean: Towards a Formalized Metatheory for the Lean Theorem Prover. Retrieved from <https://arxiv.org/abs/2403.14064>
- [7] Chris Casinghino, Vilhelm Sjöberg, and Stephanie Weirich. 2014. Combining Proofs and Programs in a Dependently Typed Language. In *Proceedings of the 41st ACM Symposium on Principles of Programming Languages (POPL)*, 2014. 33–45.
- [8] Iliano Cervesato and Frank Pfenning. 2002. A Linear Logical Framework. *Information and Computation* 179, 1 (2002), 19–75.
- [9] Robert L. Constable and Scott F. Smith. 1987. Partial Objects in Constructive Type Theory. In *Proceedings of the Second Annual IEEE Symposium on Logic in Computer Science (LICS)*, 1987. 183–193.
- [10] Thierry Coquand. 1991. An Algorithm for Testing Conversion in Type Theory. *Logical Frameworks*, 255–279.
- [11] Haskell B. Curry. 1942. The Inconsistency of Certain Formal Logics. *Journal of Symbolic Logic* 7, 3 (1942), 115–117.
- [12] Nachum Dershowitz and Zohar Manna. 1979. Proving Termination with Multiset Orderings. *Communications of the ACM* 22, 8 (1979), 465–476.
- [13] Jana Dunfield and Neel Krishnaswami. 2021. Bidirectional Typing. *ACM Computing Surveys* 54, 5 (2021), 98:1–98:38.
- [14] Qiancheng Fu and Hongwei Xi. 2023. A Two-Level Linear Dependent Type Theory. Retrieved from <https://arxiv.org/abs/2309.08673>
- [15] Eduardo Giménez. 1994. Codifying Guarded Definitions with Recursive Schemes. In *Types for Proofs and Programs (TYPES) (LNCS)*, 1994. Springer, 39–59.
- [16] Jean-Yves Girard. 1972. Interprétation fonctionnelle et élimination des coupures de l’arithmétique d’ordre supérieur. Doctoral dissertation.
- [17] Jean-Yves Girard. 1998. Light Linear Logic. *Information and Computation* 143, 2 (1998), 175–204.
- [18] V. N. Grišin. 1982. Predicate and Set-Theoretic Calculi Based on Logic Without Contractions. *Mathematics of the USSR-Izvestiya* 18, 1 (1982), 41–59.
- [19] Antonius J. C. Hurkens. 1995. A Simplification of Girard’s Paradox. In *Typed Lambda Calculi and Applications (TLCA) (LNCS)*, 1995. Springer, 266–278.
- [20] Neelakantan R. Krishnaswami, Pierre Pradic, and Nick Benton. 2015. Integrating Linear and Dependent Types. In *Proceedings of the 42nd ACM Symposium on Principles of Programming Languages (POPL)*, 2015. 17–30.
- [21] Luc Maranget. 2008. Compiling Pattern Matching to Good Decision Trees. In *Proceedings of the ACM SIGPLAN Workshop on ML*, 2008. 35–46.
- [22] Conor McBride. 2016. I Got Plenty o’ Nuttin’. *A List of Successes That Can Change the World 9600*, 207–233.
- [23] Benjamin Moon, Harley Eades III, and Dominic Orchard. 2021. Graded Modal Dependent Type Theory. In *European Symposium on Programming (ESOP) (LNCS)*, 2021. Springer, 462–490.
- [24] Leonardo de Moura and Sebastian Ullrich. 2021. The Lean 4 Theorem Prover and Programming Language. In *Automated Deduction (CADE 28) (LNCS)*, 2021. Springer, 625–635.
- [25] Frank Pfenning and Conal Elliott. 1988. Higher-Order Abstract Syntax. In *Proceedings of the ACM Conference on Programming Language Design and Implementation (PLDI)*, 1988. 199–208.
- [26] Matthieu Sozeau, Simon Boulter, Yannick Forster, Nicolas Tabareau, and Théo Winterhalter. 2020. Coq Coq Correct! Verification of Type Checking and Erasure for Coq, in Coq. *Proceedings of the ACM on Programming Languages* 4, POPL (2020), 8:1–8:28.
- [27] Sam Speight and Niels van der Weide. 2026. Impredicativity in Linear Dependent Type Theory. Retrieved from <https://arxiv.org/abs/2602.08846>
- [28] Victor Taelin. 2026. BendRT: A Parallel Runtime for CPUs and GPUs.
- [29] Kazushige Terui. 2004. Light Affine Set Theory: A Naive Set Theory of Polynomial Time. *Studia Logica* 77, (2004), 9–40.